

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to explain the three data states — at rest, in transit, and in use — and identify the encryption approach appropriate for each.
- By the end of this lecture, you will be able to describe how envelope encryption works and explain why it is the dominant pattern in cloud key management.
- By the end of this lecture, you will be able to distinguish between the three key control models — provider-managed, customer-managed, and customer-controlled — and select the appropriate model for a given regulatory and threat context.
- By the end of this lecture, you will be able to explain the roles of AES-256, TLS 1.2/1.3, asymmetric cryptography (RSA, ECC), and homomorphic encryption in a cloud encryption architecture.
- By the end of this lecture, you will be able to articulate why default cloud encryption does not constitute comprehensive data protection and what additional controls are required.

Introduction

In 2019, a breach at one of North America's largest financial institutions — Capital One — exposed the personal records of over one hundred million customers. The attacker captured IAM credentials through a server-side request forgery attack and used them to access AWS S3 buckets. The data sitting in those buckets was not encrypted. There was no cipher to break, no key to steal. Customer names, addresses, credit scores, and Social Security numbers were readable exactly as stored. That breach is not an outlier; it represents the consequence of treating encryption as an optional or deferred control rather than a baseline requirement.

Encryption is the process of converting readable data — plaintext — into an unreadable form — ciphertext — using a mathematical algorithm and a cryptographic key. The process is reversible only with the correct key, which means data intercepted, stolen, or physically extracted without the key is computationally useless to the attacker. In cloud environments, data exists in three states — at rest, in transit, and in use — and each state has a distinct encryption requirement and a distinct set of standards and implementation patterns.

This guide covers the full spectrum of cloud encryption: algorithm selection for each data state, the envelope encryption pattern that underlies most cloud key management, the three key control models that determine where governance responsibility sits, and the implementation options — client-side, server-side, and field-level encryption — available to architects. The CCSP examination tests not only the vocabulary of encryption but the practitioner's ability to reason about which approach is appropriate for a given context, and how encryption interacts with — and fails to substitute for — access controls and policy governance.

Core Concepts Explained

Three Data States and Their Encryption Requirements

Definition: Data at rest is stored data (files, database rows, disk images); its standard protection is AES-256 symmetric encryption. Data in transit is data moving across networks; its standard protection is TLS 1.2 or 1.3. Data in use is data actively being processed in memory; its emerging protection is homomorphic encryption, though practical adoption remains limited to specialized contexts.

Business relevance: Regulatory frameworks including PCI DSS, HIPAA, and the GDPR prescribe encryption for data at rest and in transit. Failure to meet these requirements exposes organizations to fines, loss of certifications, and breach liability. The three-state model ensures that no phase of a data item's lifecycle is left unprotected, addressing the full attack surface across storage, network, and compute layers.

Cloud Security Practitioner relevance: A Cloud Security Practitioner must be able to audit an architecture against all three data states, confirm that appropriate encryption is applied at each, and identify gaps — for example, an environment that encrypts volumes at rest and uses TLS for external traffic, but permits plaintext communication between internal microservices (a data-in-transit gap) or handles sensitive PII in unencrypted memory with insufficient instance isolation.

Real-world example: A healthcare analytics company stores patient MRI records in Amazon S3 (data at rest), transfers them from S3 to a processing cluster over HTTPS (data in transit), and decrypts them in memory on the compute nodes during image analysis (data in use). During a HIPAA readiness assessment, the Cloud Security Practitioner maps each phase: S3 data at rest is protected with AES-256 server-side encryption using a customer-managed KMS key — a pass. The HTTPS transfer uses TLS 1.2 with strong cipher suites — a pass. The processing cluster decrypts images into memory on EC2 instances that are multi-tenant within the account and do not use nitro-based hardware isolation with Dedicated Hosts or dedicated instances — a gap. For the most sensitive studies, the practitioner recommends Dedicated Instances to ensure no other workload can access the hypervisor layer while PHI is in memory. For less sensitive batch processing, the team documents the in-use exposure risk and implements compensating controls including strict instance-level security group rules and memory-resident sensitive data minimization (processing one record at a time, zeroing memory between records).

Risks or limitations: Failing to address data in transit allows network interception on internal paths that practitioners mistakenly assume are implicitly trusted. Failing to address data in use leaves plaintext exposed in memory during the processing window. Best practice: enforce TLS for all inter-service communication regardless of whether it is internal or external, and document any data-in-use gaps with explicit compensating controls and risk acceptance.

AES-256 for Data at Rest and TLS 1.2/1.3 for Data in Transit

Definition: AES-256 (Advanced Encryption Standard, 256-bit key) is the industry standard symmetric algorithm for data at rest. It is hardware-accelerated on modern processors and provides sufficient computational cost for the attacker that it is considered cryptographically secure for the foreseeable future. TLS (Transport Layer Security) version 1.2 and 1.3 are the current standards for data in transit; earlier versions (TLS 1.0, 1.1, SSL) carry known cryptographic weaknesses and must not be used.

Business relevance: Both standards are prescribed by the major compliance frameworks. PCI DSS 4.0 explicitly requires TLS 1.2 as a minimum. FIPS 140-2 and FIPS 140-3 approved cryptographic modules require AES-256 for data at rest in US federal and regulated financial contexts. Deploying deprecated protocols or weaker key lengths creates audit findings and may result in compliance certification failure.

Cloud Security Practitioner relevance: A Cloud Security Practitioner is responsible for ensuring that encryption configurations reflect current standards and that deprecated protocols are actively disabled, not just not preferred. TLS configuration is an ongoing operational task: certificates expire, cipher suites require periodic review, and protocol versions must be managed through the cloud provider's load balancer and API gateway configurations. Encryption algorithm selection at initial deployment is insufficient if it is not maintained.

Real-world example: A financial services company migrating to AWS discovers that its legacy internal API gateway still supports TLS 1.0 for backward compatibility with an older partner integration. The partner uses a payment processing client that was built in 2014 and was never updated to support TLS 1.2. The Cloud Security Practitioner conducts a risk assessment: TLS 1.0 is vulnerable to POODLE and other downgrade attacks, and the data transmitted (transaction authentication tokens) is high-sensitivity. The assessment recommends terminating TLS 1.0 support on a 60-day timeline: the partner is formally notified, the partner's client is updated to TLS 1.2 by the partner's team, and the API gateway enforces a minimum of TLS 1.2 on all connections. The ALB (Application Load Balancer) security policy is updated to ELBSecurityPolicy-TLS13-1-2-2021-06, which enforces TLS 1.2 as minimum and prefers TLS 1.3. After cutover, weekly automated scans using SSL Labs' API verify that no listener in the organization's cloud environment accepts connections below TLS 1.2.

Risks or limitations: Key risks include misconfigured load balancers that still accept deprecated TLS versions, certificates approaching expiry that cause service outages if not rotated, and weak cipher suite configurations that permit export-grade or otherwise compromised negotiation. Best practice: implement automated certificate management (AWS ACM, Let's Encrypt) to eliminate manual expiry risk, set minimum TLS version to 1.2 in all load balancer and API gateway configurations, and schedule annual cipher suite reviews.

Envelope Encryption and Key Management Architecture

Definition: Envelope encryption is the standard cloud key management pattern: data is encrypted with a data encryption key (DEK), and the DEK is itself encrypted ('wrapped') with a master key stored in a dedicated key management service. The encrypted DEK is stored alongside the data; the master key never leaves the key management service. Decrypting the data requires first unwrapping the DEK using the master key, then using the DEK to decrypt the data.

Business relevance: Envelope encryption solves the key distribution problem at scale: rather than encrypting all data with a single master key (which would require the master key to be widely accessible), each piece of data gets its own unique DEK. The master key is used only to wrap and unwrap DEKs, keeping it isolated in a hardware-secured KMS. Amazon S3, since early 2023, applies this pattern by default for all new objects — each object receives a unique DEK, encrypted by a master key in AWS KMS.

Cloud Security Practitioner relevance: A Cloud Security Practitioner must understand envelope encryption to make informed decisions about key management architecture. The practitioner's role includes evaluating where the master key material is held (provider infrastructure vs. customer-managed HSM), defining key rotation schedules, and auditing key usage logs to detect anomalous decryption patterns that might indicate key compromise or unauthorized access.

Real-world example: A regulated investment fund stores trade confirmation records in Amazon S3 with a compliance requirement that the fund's operations team — not AWS — controls access to the encryption key. The Cloud Security Practitioner designs an envelope encryption architecture using AWS KMS with a customer-managed key (CMK): a symmetric KMS key created in the fund's AWS account, with a key policy that permits only the compliance team's IAM role to perform kms:Decrypt and kms:GenerateDataKey operations. S3 is configured for SSE-KMS using this CMK. Each object stored in S3 receives a unique AES-256 DEK generated by KMS, encrypted with the CMK. The DEK is stored alongside the object. To decrypt any record, AWS must call KMS with the CMK — a call that is logged in CloudTrail and controlled by the key policy. If the fund's regulator

requires a key to be revoked during a regulatory hold, the fund's compliance officer can disable or schedule deletion of the CMK through the KMS console, rendering all S3 objects protected by that key inaccessible without any data movement.

Risks or limitations: Envelope encryption does not protect data if the master key is compromised or if the IAM policies governing KMS access are misconfigured to allow unauthorized parties to call kms:Decrypt. Risks also include key deletion (accidental or malicious) that permanently renders data inaccessible. Best practice: require multi-person authorization for KMS key deletion, enable KMS key rotation on a defined schedule (typically annually for AES-256), log and alert on all kms:Decrypt calls outside of expected patterns using CloudTrail and SIEM alerting.

Asymmetric Cryptography: RSA, ECC, and the TLS Handshake

Definition: Asymmetric cryptography uses a mathematically related key pair: a public key that any party can use to encrypt data, and a private key held exclusively by the owner for decryption or signing. Unlike symmetric algorithms (AES), asymmetric algorithms (RSA, Elliptic Curve Cryptography / ECC) are computationally expensive and not suited for bulk data encryption — they are used for key exchange and digital signatures.

Business relevance: TLS uses asymmetric cryptography during the handshake phase to authenticate the server (via a digital certificate) and to securely exchange a symmetric session key — then immediately switches to symmetric encryption for the data transfer itself. This hybrid model provides the security of asymmetric key exchange with the performance of symmetric bulk encryption. Every HTTPS connection a user makes relies on this pattern.

Cloud Security Practitioner relevance: A Cloud Security Practitioner must understand asymmetric cryptography well enough to evaluate certificate management practices, key length requirements (RSA 2048-bit minimum, ECC 256-bit minimum for CCSP purposes), and the role of certificate authorities in the trust chain. This matters operationally when managing public-facing TLS certificates, internal CA infrastructure for service-to-service authentication, and code signing or document integrity use cases.

Real-world example: A cloud-native startup deploys a microservices architecture on AWS ECS. Internal service-to-service communication is initially over HTTP within the private VPC network. A security assessment identifies this as a data-in-transit gap: an attacker who compromises one container can use ARP spoofing or other layer-2 techniques within the VPC to intercept internal traffic. The Cloud Security Practitioner implements mutual TLS (mTLS) for all internal service communication using AWS Private Certificate Authority: each service receives a client certificate issued by the internal CA, and the TLS handshake requires both sides to present valid certificates before the connection is established. The RSA 2048-bit certificate keys are stored in AWS Secrets Manager with automated rotation. Traffic between all services is now authenticated and encrypted, even within the private network boundary, eliminating the assumption that network co-location implies trust.

Risks or limitations: Key risks include expired certificates causing service outages, use of RSA key lengths below 2048 bits (considered weak), and reliance on public CAs for internal service authentication where an internal CA with tighter issuance controls is more appropriate. Best practice: use automated certificate management for public certificates (ACM renews automatically), implement an internal CA for service-to-service mTLS, and monitor certificate expiry across all assets with alerts 60 and 30 days before expiry.

Key Control Models: Provider-Managed, Customer-Managed, and Customer-Controlled Keys

Definition: Cloud encryption key governance is defined by three control models. Provider-managed keys: the cloud provider generates, stores, and controls the master key material entirely; the customer has no direct access to or control over the key. Customer-managed keys: the customer defines key policies and rotation within the provider's KMS, but the key material itself remains within the provider's infrastructure. Customer-

controlled keys: the customer holds and manages the master key material entirely outside the provider's systems, using their own HSM or key management infrastructure.

Business relevance: The choice of key control model is not merely a technical preference — it is a regulatory and risk governance decision. Highly regulated industries (financial services, healthcare, defense) may require customer-controlled keys to satisfy requirements that sensitive data keys never reside in third-party infrastructure. Less regulated contexts may accept provider-managed keys as a pragmatic, lower-complexity option. The model also determines the customer's ability to respond to legal orders: a provider-managed key can potentially be compelled from the provider; a customer-controlled key cannot be accessed by the provider and therefore cannot be compelled from it.

Cloud Security Practitioner relevance: A Cloud Security Practitioner must evaluate regulatory requirements, threat model, and operational capacity when recommending a key control model. Customer-controlled keys carry the highest assurance but also the highest operational burden: the customer is fully responsible for key availability, durability, and access control. Loss of a customer-controlled master key means permanent, irrecoverable loss of all data protected by it.

Real-world example: A multinational bank is architecting its cloud data strategy. Three data categories are in scope: (1) Internal employee training materials — low sensitivity, large volume, high performance requirement. Provider-managed keys (SSE-S3) are sufficient: the bank accepts the provider's key governance for non-sensitive data and prioritizes operational simplicity. (2) Customer account transaction records — regulated under PCI DSS and the bank's own data governance policy, which requires the bank to control key policy. Customer-managed keys (SSE-KMS with CMK) are selected: the bank defines key policies that restrict decryption to specific IAM roles in specific accounts, enables key rotation, and monitors KMS usage via CloudTrail. (3) Trading algorithms and proprietary pricing models — the bank's most sensitive IP, where the risk model treats the cloud provider itself as an untrusted party with potential compelled disclosure obligations. Customer-controlled keys (SSE-C / client-side encryption with HSM-held key material) are selected: the bank encrypts the data before upload using keys stored in their on-premises HSM, and the cloud provider never receives plaintext or the key. The trade-off is operational complexity and the requirement to manage the HSM infrastructure's availability and durability.

Risks or limitations: Provider-managed keys offer the least assurance against a compromised or compelled provider. Customer-controlled keys offer the highest assurance but carry the risk of permanent data loss if the key is lost or the HSM fails without a recovery plan. Best practice: match the key control model to the data classification and regulatory obligation for each dataset, document the rationale, and establish key recovery procedures for customer-managed and customer-controlled models.

Client-Side, Server-Side, and Field-Level Encryption Implementation Models

Definition: Client-side encryption: data is encrypted before it leaves the customer's environment, so the cloud provider receives and stores only ciphertext and never holds the plaintext or the key. Server-side encryption: the cloud provider encrypts data on receipt using a key under one of the three control models. Field-level encryption: specific sensitive fields within a data record (e.g., payment card numbers, government IDs) are individually encrypted, leaving the rest of the record accessible for operational use.

Business relevance: Client-side encryption provides the highest data confidentiality assurance because the provider is treated as fully untrusted, but it adds complexity to the application layer and eliminates the ability to use provider-native tools (e.g., S3 search, Athena queries) against the data without first decrypting it. Server-side encryption is simpler to implement and allows native cloud service integration. Field-level encryption is useful when regulatory requirements protect only specific data elements while operational processes need access to the broader record.

Cloud Security Practitioner relevance: A Cloud Security Practitioner selects the implementation model based on the threat model and data sensitivity. For most enterprise use cases, server-side encryption with customer-managed keys (SSE-KMS) is the appropriate baseline. Client-side encryption is reserved for highest-sensitivity data where provider trust cannot be extended. Field-level encryption is most commonly applied to cardholder data, government identifiers, and healthcare identifiers within otherwise non-sensitive records.

Real-world example: A government contractor working with classified program information uses client-side encryption before uploading to S3: the application retrieves a symmetric key from an on-premises HSM, encrypts each file locally using AES-256 GCM, and uploads the ciphertext. The S3 bucket still uses SSE-S3 as a secondary layer (encryption at the storage medium level), but the data the provider stores is already ciphertext from the client's perspective. The provider cannot read the files regardless of what compelled disclosure they might receive, because they hold only doubly encrypted data and no key. Separately, a healthcare billing company uses field-level encryption for its patient database: the diagnosis codes and treatment records are encrypted at the field level using a customer-controlled key, but patient name and address fields remain accessible to the billing system without decryption. This allows routine billing operations to proceed without exposing PHI, while the most sensitive clinical fields remain encrypted even during record access by the billing application.

Risks or limitations: Client-side encryption requires careful key management outside the cloud provider; if the local key store fails, all encrypted data is unrecoverable. Server-side encryption with provider-managed keys offers less assurance against a provider-level threat but is operationally simpler. Best practice: use server-side encryption with customer-managed KMS keys as the standard; elevate to client-side encryption only where the data classification and regulatory obligation justify the operational cost.

Practical Applications

The following scenarios show how Cloud Security Practitioners apply cloud encryption principles in architecture design, compliance assessment, and incident response contexts.

Designing a Compliant Encryption Architecture for PCI DSS Scope

A Cloud Security Practitioner is tasked with designing the encryption architecture for an e-commerce platform that will process credit card transactions and store cardholder data in AWS. The PCI DSS 4.0 requirement mandates encryption of cardholder data at rest (Requirement 3.5) and strong cryptography for all data in transit (Requirement 4.2, minimum TLS 1.2). The practitioner designs the following architecture: cardholder data at rest is stored in an RDS PostgreSQL instance with encryption enabled using a customer-managed KMS key, with annual key rotation enforced through the KMS key policy. Primary account numbers (PANs) stored in a DynamoDB table for order history use field-level encryption via AWS Encryption SDK, with PANs encrypted individually and returned only to the payment processing role — not to the order management application. All API communication between microservices uses TLS 1.2 minimum enforced through ALB security policies. The CloudFront distribution terminating customer HTTPS connections enforces TLS 1.2 and the TLSv1.2_2021 security policy, which disables all weaker cipher suites. KMS key usage is logged to CloudTrail with a SIEM alert for any kms:Decrypt call outside of the defined payment processing IAM role. The practitioner documents the encryption architecture in the PCI DSS System Security Plan and confirms with the QSA that the implementation meets scope requirements.

Evaluating Key Control Model Trade-offs for a Healthcare Cloud Migration

A healthcare system is migrating its electronic health records platform to AWS and must comply with HIPAA, which requires technical safeguards for PHI including encryption of data at rest and in transit. The Cloud Security

Practitioner leads a risk assessment to select the key control model. Provider-managed keys are evaluated first: operationally simplest, but the CISO's threat model identifies the cloud provider itself as a potential compelled disclosure risk given the sensitivity of patient data and the healthcare system's high-profile patient population. Customer-managed keys (SSE-KMS with CMK) are evaluated next: the healthcare system controls the key policy, restricting kms:Decrypt to specific IAM roles used by the EHR application, can disable or delete the key in a regulatory emergency, and receives CloudTrail audit logs for all key usage. The CISO accepts this model for the bulk of PHI storage. For the psychiatric records subset — subject to additional state-level privacy protections requiring the healthcare system to be the sole entity with key access — client-side encryption is selected: the application encrypts psychiatric records using keys from an HSM in the healthcare system's on-premises data center before upload to S3. The cloud provider stores only ciphertext. The practitioner documents the tiered approach, assigns data classification labels to each storage bucket, and maps the key control model to classification tier. Annual review is mandated to assess whether regulatory requirements have changed or whether the threat model assumptions remain valid.

Identifying and Remediating an Encryption Configuration Gap During an Audit

During a cloud security audit, a Cloud Security Practitioner reviews the encryption configuration of a company's AWS environment. The audit finds three gaps. First, an S3 bucket holding HR employee data has no server-side encryption configured — it was created before the company established its encryption standard, and the absence was never flagged. Second, an internal ECS service communicates with a backend RDS database over the VPC internal network without TLS, under the assumption that the private network is sufficiently trusted. Third, an EBS volume snapshot shared with an external consultant account was created from a volume encrypted with a CMK, but the CMK's key policy does not grant the consultant account permission to use the key — making the snapshot unreadable by the consultant and therefore the sharing arrangement non-functional, but the exposed snapshot reference still represents a misconfiguration. The practitioner addresses each gap: the S3 bucket receives SSE-KMS encryption with default encryption enabled going forward and existing objects re-encrypted via an S3 Batch Operations job; the ECS-to-RDS communication is updated to enforce SSL (the RDS PostgreSQL instance requires SSL in the parameter group, and the application connection string includes the sslmode=require flag); the EBS snapshot's sharing permission is revoked, the data for the consultant is extracted and delivered through a controlled file transfer mechanism that does not expose the snapshot itself. Each remediation is documented with a root cause and a preventive control to avoid recurrence.

Best Practices

- **Enable default encryption at the storage resource level — S3 bucket default encryption, EBS region-level encryption, RDS storage encryption — so that no resource is created without encryption regardless of individual team behavior.** Relying on individual developers or project teams to enable encryption per-resource creates systematic gaps. Cloud providers allow setting encryption defaults at the region or organization level: in AWS, enabling EBS encryption by default at the region level, setting an S3 bucket default encryption policy, and enforcing RDS encryption through SCPs means that new resources are automatically encrypted even if the requestor does not specify it. Verify these defaults are active across all regions and accounts using AWS Config conformance packs.
- **Use customer-managed KMS keys for regulated and sensitive data, and implement key policies that restrict decryption to specific IAM roles rather than granting broad account-level access.** AWS KMS key policies control who can use a key to encrypt and decrypt data. A common misconfiguration is a key policy that effectively grants any principal in the account kms:Decrypt access, rather than scoping access to the specific application role that needs it. Write key policies that explicitly permit only the necessary IAM roles

and deny all others. Enable key rotation (annual rotation for symmetric CMKs) and require multi-person authorization (MPA) for key deletion using AWS IAM conditions.

- **Enforce TLS 1.2 as the minimum protocol for all data in transit — including internal service-to-service communication within VPCs — and eliminate support for deprecated protocols on a defined timeline.** The assumption that VPC-internal traffic is implicitly trusted and does not need encryption is a persistent security gap. Attackers who compromise a single container or VM can potentially intercept internal traffic using network-level techniques. Enforce TLS at every service boundary: ALB security policies for external traffic, ACM Private CA and mTLS for internal microservice communication, and SSL/TLS enforcement flags on all database connection configurations. Automate TLS version compliance checks using AWS Config or equivalent scanning tools.
- **Implement CloudTrail logging for all KMS API calls and configure SIEM alerts for kms:Decrypt calls outside of expected patterns — such as calls from unexpected IAM identities, unusual volumes, or calls at unusual times.** Envelope encryption ensures the master key is isolated from the data, but if an attacker compromises an IAM identity with kms:Decrypt permission, they can decrypt data through the standard API without any technical alarm. Detecting this requires behavioral monitoring of KMS API activity. Establish a baseline for each CMK: which IAM roles call kms:Decrypt, at what frequency, and from which services. Alert on deviations — a sudden spike in kms:Decrypt from an infrequently used role is a higher-priority signal than high volume from the primary application role.
- **Match the key control model to the data classification and regulatory obligation for each dataset, and document the rationale with a defined review cycle.** A blanket 'use provider-managed keys for everything' or 'use customer-controlled keys for everything' policy is both over-simplified and potentially either non-compliant or operationally unmanageable. Define a data classification framework with at least three tiers (non-sensitive, sensitive/regulated, highly sensitive/confidential), map each tier to a key control model, and document why that model is appropriate given the regulatory requirements and threat assumptions for each tier. Review the mapping annually and after any significant regulatory change.

Common Mistakes and Misconceptions

Misconception: Assuming that enabling server-side encryption on an S3 bucket means the data is fully protected against unauthorized access.

Correct approach: Server-side encryption protects data at rest on the storage medium. When an authenticated API call is made using valid credentials, the provider decrypts the data on behalf of the caller and returns plaintext. Full protection requires IAM least privilege to control who can make authenticated API calls, Block Public Access to prevent unauthenticated calls, and access monitoring to detect anomalous decryption activity.

Misconception: Using TLS 1.0 or TLS 1.1 for data in transit because 'legacy clients still require it' without a defined remediation timeline.

Correct approach: TLS 1.0 and 1.1 carry known cryptographic weaknesses (POODLE, BEAST, downgrade attacks) and are prohibited under PCI DSS 4.0 and other standards. Legacy client requirements must be addressed by updating the client software on a defined timeline. Until remediation, connections using deprecated protocols should be explicitly tracked, risk-accepted at an appropriate level, and excluded from any security certification scope.

Misconception: Selecting customer-controlled keys for all data without establishing key recovery and HSM availability procedures, creating a permanent data loss risk.

Correct approach: Customer-controlled keys provide the highest assurance but require the customer to fully manage key availability and durability. Without HSM redundancy, key backup, and documented recovery procedures, loss of the HSM or the key material means permanent, irrecoverable loss of all data encrypted with that key. Implement geographically redundant HSM deployments, encrypted key backups stored in a separate secure location, and tested recovery procedures as prerequisites for any customer-controlled key deployment.

Misconception: Treating encryption as a one-time configuration decision rather than an ongoing operational responsibility requiring certificate management, key rotation, and cipher suite review.

Correct approach: Encryption configurations drift over time: certificates expire, cipher suites that were secure become deprecated, key rotation schedules lapse, and new resources are created without encryption. Implement automated certificate management (ACM), scheduled key rotation, and periodic cipher suite compliance scans. Include encryption configuration review in the change management process for any storage or communication infrastructure change.

table: Cloud Encryption Decision Framework

Use this framework to select encryption approach, algorithm, and key control model for any cloud data asset. Work through all three columns for each dataset in scope.

Data State / Use Case	Algorithm / Protocol	Key Control Guidance
Data at rest — non-sensitive	AES-256 (SSE-S3 / provider-managed)	Provider-managed keys acceptable; lowest operational overhead
Data at rest — regulated (PCI, HIPAA)	AES-256 (SSE-KMS, CMK)	Customer-managed KMS key; define key policy, enable rotation, log usage
Data at rest — highly confidential / IP	AES-256 (client-side, customer-controlled)	Customer-controlled HSM key; provider never receives plaintext; HSM HA required
Specific sensitive fields (PAN, PHI ID, SSN)	AES-256 field-level via Encryption SDK / similar	Customer-managed key scoped to encryption/decryption role; field-level key policy
Data in transit — external (user-facing)	TLS 1.2 or 1.3 (HTTPS, enforce via ALB policy)	PKI managed by ACM; auto-renewal; TLSv1.2_2021 security policy minimum
Data in transit — internal (service-to-service)	TLS 1.2 or 1.3 (mTLS preferred)	Private CA (ACM PCA); mutual certificate authentication; automated rotation
Key exchange / digital signatures	RSA 2048-bit minimum, ECC 256-bit minimum	Public/private key pair; never expose private key; rotate at certificate renewal
Data in use — limited adoption context	Homomorphic encryption (experimental)	Customer-controlled; computational overhead significant; evaluate per-use-case

How to use this: Every row with 'customer-managed' or 'customer-controlled' keys requires a documented key policy, rotation schedule, and recovery procedure before deployment.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A financial institution stores customer tax records in Amazon S3. The institution's compliance team requires that the institution — not AWS — control the key material used to encrypt the data, but the institution is willing to have the key stored within AWS KMS infrastructure. Which key control model meets this requirement?

- A) A. Provider-managed keys (SSE-S3), because AWS generates and manages the key within its infrastructure.
- B) B. Customer-managed keys (SSE-KMS with CMK), because the institution defines the key policy and controls who can use the key, while the key material remains in AWS KMS.
- C) C. Customer-controlled keys (SSE-C), because the institution provides its own key material from an external HSM and AWS never stores the key.
- D) D. Envelope encryption is not applicable to this scenario because the data must be encrypted at the field level.

Answer: B. Customer-managed keys (SSE-KMS with CMK), because the institution defines the key policy and controls who can use the key, while the key material remains in AWS KMS.

Q2. Which protocol version must be enforced as the minimum for data in transit under PCI DSS 4.0, and why are earlier versions prohibited?

- A) A. TLS 1.0, which is the minimum acceptable version for backward compatibility with legacy payment clients.
- B) B. TLS 1.2, because TLS 1.0 and 1.1 have known vulnerabilities including POODLE and downgrade attacks and are explicitly prohibited by PCI DSS 4.0.
- C) C. TLS 1.3 exclusively, because TLS 1.2 is now deprecated under all major compliance frameworks.
- D) D. SSL 3.0 or higher, as it remains acceptable for internal communications not exposed to the internet.

Answer: B. TLS 1.2, because TLS 1.0 and 1.1 have known vulnerabilities including POODLE and downgrade attacks and are explicitly prohibited by PCI DSS 4.0.

Q3. An organization uses AWS S3 with server-side encryption (SSE-KMS) for all stored data. An attacker compromises an IAM role that has kms:Decrypt and s3:GetObject permissions. What is the most accurate description of the data confidentiality impact?

- A) A. The attacker cannot access plaintext data because SSE-KMS encryption prevents decryption without the root account's consent.
- B) B. The attacker can access plaintext data by calling the S3 and KMS APIs using the compromised role's credentials, because the role has the permissions required for decryption.
- C) C. The attacker can only access encrypted ciphertext because KMS keys require multi-factor authentication for each decryption operation.
- D) D. The attacker is blocked by KMS key rotation, which invalidates previously issued decryption permissions after 90 days.

Answer: B. The attacker can access plaintext data by calling the S3 and KMS APIs using the compromised role's credentials, because the role has the permissions required for decryption.

Q4. In the envelope encryption model, what is the primary purpose of the data encryption key (DEK)?

- A) A. The DEK is the master key stored in KMS that protects all encrypted data in the account.
- B) B. The DEK is used to directly encrypt and decrypt data, while itself being encrypted ('wrapped') by the master key stored in the KMS.
- C) C. The DEK is a session token that grants temporary access to the KMS master key for one decryption operation.
- D) D. The DEK is the asymmetric public key used during the TLS handshake to establish a secure session for key exchange.

Answer: B. The DEK is used to directly encrypt and decrypt data, while itself being encrypted ('wrapped') by the master key stored in the KMS.

Q5. A government contractor needs to store highly sensitive program information in AWS S3 with the requirement that AWS never holds the encryption key and cannot access the data under any circumstances, including compelled legal disclosure. Which implementation model satisfies this requirement?

- A) A. SSE-S3 with provider-managed keys, because AWS implements strong internal controls that prevent unauthorized access.
- B) B. SSE-KMS with a customer-managed key, because the institution controls the key policy within AWS KMS.
- C) C. Client-side encryption using keys from an on-premises HSM, because the data is encrypted before upload and AWS receives only ciphertext and never holds the key material.
- D) D. SSE-KMS with key rotation enabled, because frequent rotation prevents AWS from accessing any single version of the data.

Answer: C. Client-side encryption using keys from an on-premises HSM, because the data is encrypted before upload and AWS receives only ciphertext and never holds the key material.

Reflection Questions

1. Review the three data states against your current or target cloud environment: can you confirm that appropriate encryption is applied at each state, or are there gaps — particularly for data in transit between internal services?
2. Which key control model does your organization currently use for its most sensitive data? Does that model align with your regulatory obligations and threat assumptions, or was it selected primarily for convenience?
3. The Capital One breach involved unencrypted data in S3. Has your organization confirmed that all production S3 buckets have encryption enabled, and is there a policy that prevents new buckets from being created without it?
4. Do you know when the KMS keys protecting your most critical datasets last had their rotation settings reviewed? What would trigger an alert if a key were disabled or deleted unexpectedly?
5. Consider a scenario where your cloud provider received a legal order for access to your data: would your current key control model allow or prevent that access? Is that the right answer for your organization's risk profile?

Key Takeaways

- Data exists in three states — at rest, in transit, and in use — and each requires a distinct encryption approach: AES-256 at rest, TLS 1.2/1.3 in transit, and emerging homomorphic approaches for in use.
- AES-256 is hardware-accelerated and the industry standard for data at rest; TLS 1.2 and 1.3 are current standards for data in transit, with all earlier versions prohibited under major compliance frameworks.
- Envelope encryption is the standard cloud key management pattern: a unique DEK encrypts each data item, and the DEK is wrapped by a master key in a dedicated KMS — isolating the master key from the data at scale.
- The three key control models — provider-managed, customer-managed, and customer-controlled — represent increasing levels of assurance and operational burden; selection should be driven by data classification and regulatory obligation.
- Client-side encryption ensures the cloud provider never receives plaintext or key material; server-side encryption is simpler but extends trust to the provider's key management; field-level encryption protects specific sensitive attributes within otherwise accessible records.

- Default cloud encryption (e.g., S3 SSE since 2023) is a necessary baseline but does not constitute comprehensive protection — IAM least privilege, access monitoring, and key policy governance are equally required.
- TLS must be enforced for all communications, including internal VPC service-to-service traffic, not just external-facing connections — the assumption that private network co-location implies trust is a persistent security gap.
- Key management is an ongoing operational responsibility: certificates must be monitored for expiry, keys must be rotated on schedule, cipher suites must be reviewed periodically, and KMS API activity must be monitored for anomalous decryption patterns.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Confirm this week that all S3 buckets in your cloud environment have default encryption enabled with at least SSE-S3, and that no buckets are missing encryption — use AWS Config's s3-bucket-server-side-encryption-enabled rule.
7. Review your cloud environment's load balancer and API gateway TLS configuration: confirm that TLS 1.2 is enforced as the minimum and that no listener accepts TLS 1.0 or 1.1 connections.
8. Identify the key control model in use for your three most sensitive datasets and verify that the model matches your data classification policy; open a remediation ticket for any mismatches.
9. Enable CloudTrail KMS data event logging if not already active and configure a SIEM alert for kms:Decrypt calls from IAM identities not in the expected baseline for each customer-managed key.